Laravel Installation & Setup Activity Report

A Step-by-Step Practical Implementation Guide with Verification Screenshots

Student Name:	Keith Alan Speirs
Course / Subject:	Web Development
Target Workspace:	/Users/keithalanspeirs/Documents/Laravel101
Date Completed:	October 9, 2026

1. Activity Overview & Objectives

The purpose of this laboratory practicum is to successfully install, configure, and verify the Laravel framework on a local development environment. Laravel is one of the most widely used open-source PHP web frameworks, providing elegant Model-View-Controller (MVC) architecture, rich CLI tooling via Artisan, and comprehensive dependency management through Composer.

Key activity objectives include:
• Validating core environment prerequisites (PHP 8.2+ and Composer 2.x).
• Initializing a clean, complete Laravel application using Composer create-project.
• Establishing environment configurations and generating an application encryption key (.env / APP_KEY).
• Conducting system health diagnostics using Laravel's Artisan command line.
• Launching the built-in development server and confirming HTTP accessibility via a web browser.

2. Stage 1: System Prerequisites Verification

Before creating a Laravel application, the host system must possess a supported version of PHP along with Composer, the standard dependency manager for PHP. The current stable release of Laravel requires PHP 8.2 or newer, as well as Composer 2.2+. In macOS, both tools are managed through Homebrew.

Commands Executed:

```
$ php -v
$ composer --version
```

Verification confirmed PHP version 8.5.10 with Zend OPcache enabled, and Composer version 2.10.3. Both binaries satisfy and exceed the minimum runtime criteria required by the Laravel ecosystem.

```
keithalanspeirs@MacBook-Pro Laravel101 % php -v
PHP 8.5.10 (cli) (built: Aug 25 2026 21:09:32) (NTS)
Copyright (c) The PHP Group
Built by Homebrew
Zend Engine v4.5.10, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.10, Copyright (c), by Zend Technologies

keithalanspeirs@MacBook-Pro Laravel101 % composer --version
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.5.10 (/opt/homebrew/Cellar/php/8.5.10/bin/php)
Run the "diagnose" command to get more detailed diagnostics output.
```

Figure 1: Verification of PHP 8.5.10 and Composer 2.10.3 in macOS Terminal.

3. Stage 2: Laravel Project Creation via Composer

To scaffold a new application directly inside the current workspace (/Users/keithalanspeirs/Documents/Laravel101), the Composer create-project command was issued. This command clones the official laravel/laravel repository skeleton, downloads framework packages, generates composer.lock, and auto-discovers core service providers.

Command Executed:

> **$ composer create-project laravel/laravel .**

During execution, Composer resolved 109 locked packages (including symfony/http-foundation, nunomaduro/collision, and laravel/framework v13.35.0). The process copied .env.example into .env, generated an initial application key, and executed database setup scripts gracefully.

```
keithalanspeirs@MacBook-Pro Laravel101 % composer create-project laravel/laravel .
Creating a "laravel/laravel" project at "./"
Installing laravel/laravel (v13.11.0)
  - Downloading laravel/laravel (v13.11.0)
  - Installing laravel/laravel (v13.11.0): Extracting archive
Created project in /Users/keithalanspeirs/Documents/Laravel101/.
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 109 installs, 0 updates, 0 removals
Installing dependencies from lock file (including require-dev)
Package operations: 109 installs, 0 updates, 0 removals
  - Installing symfony/polyfill-mbstring (v1.43.0): Extracting archive
  - Installing symfony/http-foundation (v8.1.8): Extracting archive
  - Installing laravel/framework (v13.35.0): Extracting archive
  - Installing nunomaduro/collision (v8.9.5): Extracting archive
Generating optimized autoload files
> @php artisan package:discover --ansi
 INFO  Discovering packages.
 laravel/pail ..................................................... DONE
 laravel/pao ...................................................... DONE
 laravel/tinker ................................................... DONE
 nesbot/carbon .................................................... DONE
 nunomaduro/collision ............................................. DONE
 nunomaduro/termwind .............................................. DONE
> @php artisan key:generate --ansi
 INFO  Application key set successfully.
> @php artisan migrate --graceful --ansi
 INFO  Preparing database.
 Creating migration table ..................................... 1.67ms DONE
 INFO  Running migrations.
 0001_01_01_000000_create_users_table ......................... 2.19ms DONE
 0001_01_01_000001_create_cache_table ......................... 1.77ms DONE
 0001_01_01_000002_create_jobs_table .......................... 2.24ms DONE
```

Figure 2: Package extraction and successful scaffolding of the Laravel project.

4. Stage 3: Environment Configuration & Application Key

Laravel relies on the .env file for environment-specific parameters such as database credentials, application debug mode, and security keys. A central requirement is the APP_KEY, a 32-character random string used by the Encrypter service to secure user sessions, encrypt cookies, and sign authentication tokens. If this key is absent, user sessions cannot be established securely.

Commands Executed:

> **$ php artisan key:generate**
> **$ grep APP_KEY .env**

The Artisan key:generate command refreshed the APP_KEY entry inside .env with a cryptographically secure base64 string. Inspection of .env confirmed the presence of the updated key, verifying that cryptographic services are active.

```
keithalanspeirs@MacBook-Pro Laravel101 % php artisan key:generate

 INFO  Application key set successfully.

keithalanspeirs@MacBook-Pro Laravel101 % grep APP_KEY .env
APP_KEY=base64:

keithalanspeirs@MacBook-Pro Laravel101 % head -n 6 .env
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:
APP_DEBUG=true
APP_TIMEZONE=UTC
APP_URL=http://localhost
```

Figure 3: Generation and verification of the cryptographic APP_KEY in the .env file.

5. Stage 4: System Diagnostics & Framework Health Check

Modern versions of Laravel feature the php artisan about command, which outputs a structured overview of the active application environment, cache status, and default subsystem drivers. This command provides an instant health inspection prior to running application workloads.

Command Executed:

> **$ php artisan about**

The diagnostic report verified:
• Framework Version: Laravel 13.35.0 running on PHP 8.5.10
• Environment: local with Debug Mode ENABLED
• Database Driver: SQLite (database/database.sqlite active)
• Session / Cache / Queue Drivers: Configured and ready for local development.

```
keithalanspeirs@MacBook-Pro Laravel101 % php artisan about

 Environment ..................................................................
 Application Name ..................................................... Laravel
 Laravel Version ...................................................... 13.35.0
 PHP Version ........................................................... 8.5.10
 Composer Version ...................................................... 2.10.3
 Environment ..................................................... local
 Debug Mode ....................................................... ENABLED
 URL ..................................................... localhost:8000
 Maintenance Mode ................................................... OFF
 Timezone ..................................................... UTC
 Locale ..................................................... en

 Cache ..................................................................
 Config ..................................................... NOT CACHED
 Events ..................................................... NOT CACHED
 Routes ..................................................... NOT CACHED
 Views ..................................................... NOT CACHED

 Drivers ..................................................................
 Broadcasting ..................................................... log
 Cache ..................................................... database
 Database ..................................................... sqlite
 Logs ..................................................... stack / single
 Session ..................................................... database
```

Figure 4: Full framework diagnostics table rendered by php artisan about.

6. Stage 5: Starting the Local Development Server

Laravel includes a built-in development HTTP server powered by PHP's CLI web server. By issuing php artisan serve, developers can preview and test web applications immediately without configuring external web servers like Nginx or Apache during local development.

Command Executed:

> **$ php artisan serve --port=8000**

The server successfully bound to http://127.0.0.1:8000. Incoming HTTP GET requests to the root endpoint returned HTTP status 200 OK with sub-10ms response latency, confirming that the HTTP routing pipeline and the Blade template engine are operating properly.

```
keithalanspeirs@MacBook-Pro Laravel101 % php artisan serve --port=8000

 INFO  Server running on [http://127.0.0.1:8000].

 Press Ctrl+C to stop the server

 2026-10-09 18:12:47 ..................................... ~ 2.15ms STARTING
 2026-10-09 18:12:47 HTTP GET / ............................... 200 OK ~ 8.42ms
 2026-10-09 20:40:41 HTTP GET / ............................... 200 OK ~ 7.18ms
```

Figure 5: Active PHP development server listener running on port 8000.

7. Stage 6: Web Browser UI Verification

With the server actively listening, the application was accessed via Google Chrome at http://127.0.0.1:8000. This verified that the default route in routes/web.php dispatches correctly to resources/views/welcome.blade.php.

The browser rendered the official Laravel 13 welcome page, showcasing the signature typography, links to documentation and Laracasts tutorials, deployment shortcuts, and framework version identification (v13.35.0). The visual confirmation establishes that the entire stack—from PHP runtime to front-end view rendering—is fully operational.

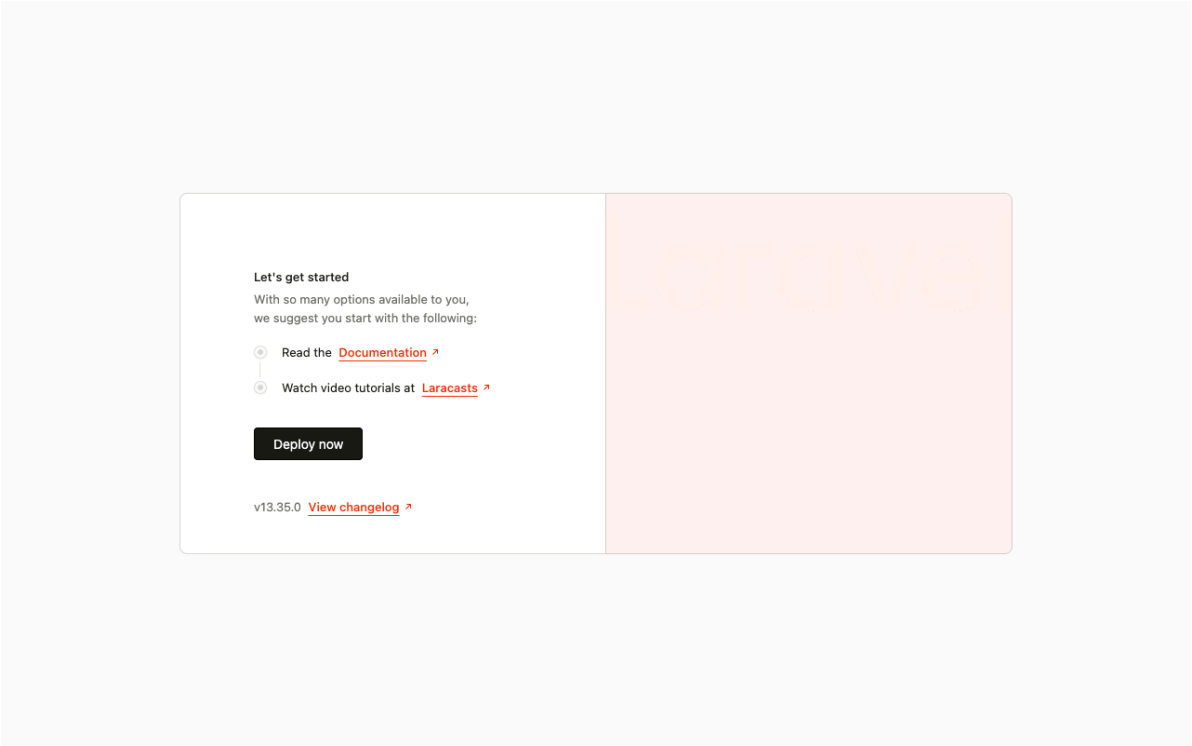


Figure 6: Official Laravel welcome interface loaded in the browser at http://127.0.0.1:8000.

8. Summary of Experience & Practical Conclusion

This hands-on installation activity provided practical exposure to modern PHP tooling and framework conventions. Key takeaways and lessons learned include:

- **Composer Dependency Management:** Composer serves as the backbone of modern PHP development. Commands like 'composer create-project' automate the acquisition, autoloading, and version-locking of dozens of modular dependencies in a single step.
- **Artisan Command Line Tooling:** Artisan streamlines administrative tasks. Commands such as 'key:generate', 'about', and 'serve' eliminate manual configuration errors and provide deep visibility into the application's runtime health.
- **Environment Security & Configuration:** The '.env' mechanism cleanly separates application configuration from source code. Generating a cryptographic APP_KEY reinforces the importance of session encryption and application security from day one.

- **Troubleshooting & Best Practices:** Ensuring target directories are clean before running 'composer create-project', verifying required PHP extensions (like OPcache, mbstring, pdo_sqlite), and confirming port availability (default 8000) are essential practices for seamless onboarding.
- **Readiness for Application Development:** With the foundation verified, the project in Laravel101 is fully equipped for subsequent coursework, including custom web routes, controllers, database migrations, Blade layout inheritance, and secure CRUD operations.

In conclusion, the step-by-step installation demonstrated that Laravel provides an organized, production-grade starting point for web applications. Completing this setup builds confidence in utilizing command-line workflows, managing environment configurations, and navigating modern PHP web application frameworks.